TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE · 3ER AÑO

Cátedra: Desarrollo de Aplicativos Móviles · Ciclo Lectivo 2026

AUDITORÍA TÉCNICA OFICIAL & ALIGNMENT

INFORME TÉCNICO DE ARQUITECTURA Y ESTADO

Auditoría de alineación del proyecto frente al programa oficial de la materia (Semanas 1 a 4)

PROYECTO AUDITADO

miPrimeraAPPMobile ("La Tienda")

PILA TECNOLÓGICA

React Native 0.86 · Expo SDK 57 · TypeScript

ROL DEL AUDITOR

Ingeniero de Software Mobile Principal & Arquitecto de Soluciones

DICTAMEN DE CUMPLIMIENTO

DESALINEACIÓN CRÍTICA DE DOMINIO Y ALCANCE

Condición operativa: Modo de solo lectura estricto (cero modificaciones en archivos del proyecto)

Fecha de emisión: 19 de Septiembre de 2026

1. Resumen Ejecutivo y Diagnóstico Principal

El presente informe surge de la auditoría del repositorio miPrimeraAPPMobile frente al programa curricular oficial de la cátedra Desarrollo de Aplicativos Móviles (Clase 1: Programa y Encuadre, Encuentro 2: Setup Expo, Semana 3: De React a React Native y Semana 4: Layout Avanzado, Imágenes y Listas).

& þ ALERTA DE ARQUITECTURA: DESVIACIÓN CRÍTICA DE DOMINIO

El código actual implementa un "Panel Cybernetic OS // V2.4" con módulos de ciberseguridad (FIREWALL NEURAL, ENLACE SATELITAL, etc.) y un buscador de texto.
El requerimiento formal de la cátedra para las Semanas 3 y 4 es el Proyecto Integrador "La Tienda": un catálogo comercial con 8 productos cotidianos (Auriculares, Mochila, Zapatillas, etc.), un Hero adaptativo con useWindowDimensions y una lista FlatList sin navegación entre pantallas.

2. Matriz Comparativa: Requerimientos vs. Estado Actual

Eje de Evaluación	Requerimiento Oficial (PDFs)	Estado en Repositorio	Dictamen
Dominio / Negocio	E-commerce ("La Tienda" con productos cotidianos)	Panel Cyberpunk ("Firewall Neural", "Enlace Satelital")	NO CUMPLE
Alcance Temporal	Hito Semanas 3-4: 1 pantalla, 1 solo scroll, sin navegación	Navegación por Tabs activas y ruta secundaria /explore	DESALINEADO
Contrato de Datos	Array de 8 productos: id, nombre, precio, categoria, imagen	Array de 6 ítems: id, nombre, categoria, estado, nivel	NO CUMPLE
Componente Tarjeta	ProductCard (row, imagen 96x96, precio local es-AR, sombra dual)	ModuleCard (columna, badges de estado, Alert genérico)	NO CUMPLE
Cabecera y Hero	ListHeaderComponent con Eyebrow, Título, Contador y Hero adaptativo	Cabecera simple con TextInput de búsqueda y botón a /explore	NO CUMPLE
Responsividad	useWindowDimensions() para altura de Hero y padding reactivo	Paddings fijos; sin cálculo reactivo de dimensiones	NO CUMPLE
Configuración FlatList	ItemSeparatorComponent (14px), ListEmptyComponent y contentContainer	FlatList elemental con márgenes manuales en tarjetas	INCOMPLETO
Assets de Medios	hero.jpg y 8 imágenes estáticas locales en assets/	Solo iconos base del template de Expo	NO CUMPLE

3. Hoja de Ruta Curricular del Proyecto Integrador (15 Semanas)

El programa de la materia estructura la formación a través de un único proyecto integrador ("La Tienda") que evoluciona semana a semana. Es fundamental respetar el hito de cada bloque sin adelantarse a conceptos posteriores:

SEMANAS 1 Y 2 · BLOQUE 1

Fundamentos y Panorama Mobile

Setup de Expo, ejecución en Expo Go, estructura básica, repositorios Git e introducción a componentes core.

SEMANAS 3 Y 4 · BLOQUE 2 [FASE ACTUAL]

Componentes Core, Flexbox y Listas con FlatList

Tarjeta de producto con StyleSheet y Pressable, cálculo de dimensiones con useWindowDimensions, cabecera con ImageBackground y lista virtualizada con 8 productos. Sin APIs ni navegación.

SEMANAS 5 A 8 · BLOQUE 3

Estado, Navegación y Datos Externos

Formularios táctiles, navegación formal con Expo Router (Listado a Detalle), peticiones HTTP a API REST real y persistencia local del carrito con AsyncStorage.

SEMANA 9 · BLOQUE 4

Evaluación Parcial

Parcial teórico-práctico integrador sobre los Bloques 1 a 3 y recuperatorio.

SEMANAS 10 A 12 · BLOQUE 5

Gestos, Acceso a Hardware y Estado Global

Gestos y drag & drop, captura de fotos con la cámara nativa, geolocalización y estado global entre pantallas con Context API.

SEMANAS 13 A 15 · BLOQUE 6

Cierre, Compilación en la Nube y Defensa Final

Build de producción real con EAS Build, instalación en dispositivos físicos y defensa oral de decisiones técnicas.

4. Arquitectura Canónica vs. Repositorio Actual

Estructura Requerida por la Cátedra

```
mi-tienda/
% % %  App.js (exporta <Products />)
% % %  components/
%    % % %  ProductCard.jsx
% % %  screens/
%    % % %  Products.jsx (FlatList)
% % %  assets/
    % % %  hero.jpg
    % % %  [8 productos].jpg

» Arquitectura plana desacoplada
» Sin dependencias de enrutador
```

Estructura Actual en el Repositorio

```
miPrimeraAPPMobile/
% % %  src/
%    % % %  app/
%    %    % % %  _layout.tsx (Tabs)
%    %    % % %  index.tsx (Cyberpunk)
%    %    % % %  explore.tsx
%    % % %  components/ui/ModuleCard.tsx
%    % % %  constants/theme.ts
% % %  assets/tabIcons/

» Plantilla moderna Expo Router v57
» Tabs activas (Semana 6 adelantada)
```

' SOLUCIÓN DE INTEGRACIÓN SIN CONFLICTOS DE ENTORNO

Para no desconfigurar Expo 57, se mantendrá src/app/index.tsx como punto de renderizado de la pantalla Products, modularizando ProductCard.tsx dentro de src/components/ y desactivando temporalmente la barra de tabs en _layout.tsx.

5. Especificación Técnica Canónica del Código Requerido

A continuación se documentan los contratos técnicos canónicos extraídos de las diapositivas de la Semana 4:

A. Contrato de Datos de los 8 Productos (Slide 19, 29 y 30)

```
const products = [
  { id: 1, nombre: "Auriculares Air", precio: 89900, categoria: "Tecnología", imagen: require("../assets/
headphones.jpg") },
  { id: 2, nombre: "Mochila Urban", precio: 64500, categoria: "Accesorios", imagen: require("../assets/
backpack.jpg") },
  { id: 3, nombre: "Zapatillas Move", precio: 112000, categoria: "Calzado", imagen: require("../assets/
sneakers.jpg") },
  { id: 4, nombre: "Botella Térmica", precio: 32900, categoria: "Hogar", imagen: require("../assets/bottle.jpg") },
  { id: 5, nombre: "Lámpara Desk", precio: 54900, categoria: "Hogar", imagen: require("../assets/lamp.jpg") },
  { id: 6, nombre: "Teclado Compact", precio: 78500, categoria: "Tecnología", imagen: require("../assets/
```

B. Componente Reutilizable: ProductCard.jsx (Slides 26 a 28)

```
export default function ProductCard({ id, nombre, precio, categoria, imagen }) {
  return (
    <Pressable style={({ pressed }) => [styles.card, pressed && styles.pressed]}>
      <Image source={imagen} style={styles.image} resizeMode="cover" />
      <View style={styles.info}>
        <Text style={styles.category}>{categoria}</Text>
        <Text style={styles.name}>{nombre}</Text>
        <Text style={styles.price}>$ {precio.toLocaleString("es-AR")}</Text>
      </View>
    </Pressable>
  );
}
const shadow = Platform.select({
  android: { elevation: 4 },
```

C. Pantalla Principal: Products.jsx (Slides 31 a 33)

```
const { width } = useWindowDimensions();
const compact = width < 380;
const heroHeight = Math.max(180, Math.min(300, width * 0.42));
const horizontalPadding = compact ? 16 : 24;

<FlatList
  data={products}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => <ProductCard {...item} />}
  contentContainerStyle={[styles.content, { paddingHorizontal: horizontalPadding }]}
  ItemSeparatorComponent={() => <View style={styles.separator} />}
  ListEmptyComponent={<Text style={styles.empty}>No hay productos disponibles.</Text>}
  ListHeaderComponent={
    <>
      <Header title="Productos" count={`${products.length} artículos`} eyebrow="MI TIENDA" />
      <ImageBackground source={require("../assets/hero.jpg")} style={[styles.hero, { height: heroHeight }]}>
        <HeroOverlay title="Elegí tu próximo favorito" subtitle="Diseño simple. Productos útiles." />
      </ImageBackground>
      <Text style={styles.sectionTitle}>Destacados</Text>
```

6. Checklist de Validación y Criterios de Cierre (Slide 35)

- Componentes desacoplados: ProductCard en archivo independiente.
- Ninguna navegación activa: Vista autocontenida de pantalla única.
- Lista con FlatList: Virtualización nativa implementada.
- Claves únicas y string: keyExtractor={(item) => String(item.id)}.
- Imágenes visibles: Dimensiones fijas definidas (96x96 en tarjetas).
- Scroll funcional unificado: ListHeaderComponent evita doble scroll.
- Estilos adaptables: useWindowDimensions() para hero y paddings.
- Sin errores ni warnings: Compilación TypeScript 100% limpia.